

Desafio 01 - P3 - 14/09/2026

Desafio escolhido: 2 - Sistema de Estoque — TechParts

Integrantes:

Flávio Henrique
Gabriel Santos
Gabriel Pereira
Gustavo Almeida
Kauã dos Santos
Leandro Bispo
Lucas Eiji
Luiz Fernando

Explicação:

O código funciona como um sistema simples de estoque. Cada peça é armazenada em uma lista no formato código, nome, preço, quantidade, permitindo alterar a quantidade diretamente sem recriar o cadastro. A árvore binária de busca armazena a mesma lista da peça e permite encontrá-la pelo código sem precisar percorrer todo o estoque. A listagem usa o Insertion Sort que organiza uma cópia do estoque por preço, mantendo a lista original sem alterações. O main() controla o menu e permite cadastrar, atualizar, listar e buscar peças.

Código resolução:

```
class NoArvore:
```

```
    # cria o no da arvore que vai guardar os dados da peca
```

```
    def __init__(self, peca):
```

```
        self.peca = peca
```

```
        self.esquerda = None
```

```
        self.direita = None
```

```
class ArvoreBuscaCodigo:
```

```
# arvore para buscar a peca pelo codigo de um jeito mais rapido
```

```
def __init__(self):
```

```
    self.raiz = None
```

```
def inserir(self, peca):
```

```
    # se a arvore estiver vazia a peca vira a raiz
```

```
    if self.raiz is None:
```

```
        self.raiz = NoArvore(peca)
```

```
    else:
```

```
        # se ja tem raiz chama a funcao para procurar onde colocar
```

```
        self._inserir_recursivo(self.raiz, peca)
```

```
def _inserir_recursivo(self, nodo_atual, peca):
```

```
    codigo_novo = peca[0]
```

```
    codigo_atual = nodo_atual.peca[0]
```

```
    # se o codigo for menor vai para o lado esquerdo
```

```
    if codigo_novo < codigo_atual:
```

```
        if nodo_atual.esquerda is None:
```

```
            nodo_atual.esquerda = NoArvore(peca)
```

```
        else:
```

```
            self._inserir_recursivo(nodo_atual.esquerda, peca)
```

```
    # se for maior vai para o lado direito
```

```
    elif codigo_novo > codigo_atual:
```

```
        if nodo_atual.direita is None:
```

```
            nodo_atual.direita = NoArvore(peca)
```

```
        else:
```

```
            self._inserir_recursivo(nodo_atual.direita, peca)
```

```
def buscar(self, codigo):  
    # começa a procurar a partir da raiz  
    return self._buscar_recursivo(self.raiz, codigo)  
  
def _buscar_recursivo(self, nodo_atual, codigo):  
    # se não achar nada retorna nulo  
    if nodo_atual is None:  
        return None  
  
    codigo_atual = nodo_atual.pecas[0]  
    # se achou o código devolve a peça  
    if codigo == codigo_atual:  
        return nodo_atual.pecas  
  
    # se o código que procuro é menor vou pra esquerda  
    elif codigo < codigo_atual:  
        return self._buscar_recursivo(nodo_atual.esquerda, codigo)  
  
    # senão vou pra direita  
    else:  
        return self._buscar_recursivo(nodo_atual.direita, codigo)  
  
def insertion_sort_por_preco(estoque_original):  
    # copia a lista para não mexer na original  
    lista = estoque_original.copy()  
  
    # passa por todas as peças a partir da segunda  
    for i in range(1, len(lista)):
```

```

chave = lista[i]

j = i - 1

# o indice 2 eh o preco. empurra as pecas mais caras pra frente
while j >= 0 and chave[2] < lista[j][2]:

    lista[j + 1] = lista[j]

    j -= 1

# bota a peca no lugar certo dela
lista[j + 1] = chave

return lista

```

```

def atualizar_quantidade(arvore, codigo, nova_quantidade):

    # usa a arvore pra achar a peca rapidinho
    peca = arvore.buscar(codigo)

    if peca:

        # o indice 3 eh a quantidade. muda direto na mesma lista
        peca[3] = nova_quantidade

        return True

    return False

```

```

def main():

    # lista que guarda tudo e a arvore pra buscas
    estoque = []

    arvore_indices = ArvoreBuscaCodigo()

    # loop infinito pro menu funcionar ate mandar sair
    while True:

```

```
print("\n" + "="*40)

print(" SISTEMA DE ESTOQUE - TECHPARTS")

print("="*40)

print("1. Cadastrar nova peça")

print("2. Atualizar quantidade em estoque")

print("3. Listar estoque (Ordenado por Preço)")

print("4. Buscar peça por código")

print("5. Sair")


opcao = input("Escolha uma opção: ")


if opcao == '1':

    try:

        codigo = int(input("Código da peça (numérico): "))

        # olha na arvore se o codigo ja existe antes de cadastrar

        if arvore_indices.buscar(codigo) is not None:

            print("Erro: Já existe uma peça cadastrada com este código.")

            continue


        nome = input("Nome da peça: ")

        preco = float(input("Preço: R$ "))

        quantidade = int(input("Quantidade em estoque: "))


        # cria a peca como lista e nao tupla pra poder mudar depois

        nova_peca = [codigo, nome, preco, quantidade]


        # guarda no estoque e na arvore ao mesmo tempo

        estoque.append(nova_peca)
```

```

    arvore_indices.inserir(nova_peca)

    print("Peça cadastrada com sucesso!")
except ValueError:
    print("Erro: O código, preço e quantidade devem ser números válidos.")

elif opcao == '2':
    try:
        codigo = int(input("Código da peça a ser atualizada: "))
        nova_qtd = int(input("Nova quantidade total em estoque: "))

        # chama a funcao que muda a quantidade
        sucesso = atualizar_quantidade(arvore_indices, codigo, nova_qtd)

        if sucesso:
            print("Quantidade atualizada com sucesso! (Registro alterado in-place)")
        else:
            print("Erro: Peça não encontrada pelo código.")
    except ValueError:
        print("Erro: Código e quantidade devem ser números inteiros.")

elif opcao == '3':
    if not estoque:
        print("O estoque está vazio.")
    else:
        # chama o insertion sort pra ordenar so na hora de mostrar
        estoque_ordenado = insertion_sort_por_preco(estoque)

        print("\n--- Catálogo Ordenado por Preço ---")

```

```

print(f'{'CÓDIGO':<8} | {'NOME':<20} | {'PREÇO':<10} | {'QTD':<5}')

print("-" * 52)

for p in estoque_ordenado:

    print(f"{p[0]:<8} | {p[1]:<20} | R$ {p[2]:<7.2f} | {p[3]:<5}")

elif opcao == '4':

    try:

        codigo = int(input("Digite o código da peça que deseja buscar: "))

        # usa a arvore pra tentar achar a peca

        peca = arvore_indices.buscar(codigo)

    if peca:

        print("\n--- Peça Encontrada ---")

        print(f"Código: {peca[0]}")

        print(f"Nome: {peca[1]}")

        print(f"Preço: R$ {peca[2]:.2f}")

        print(f"Quantidade: {peca[3]} unidades")

    else:

        print("Peça não encontrada no catálogo.")

    except ValueError:

        print("Erro: Código inválido.")

elif opcao == '5':

    # sai do loop e acaba o programa

    print("Encerrando o sistema da TechParts...")

    break

else:

    print("Opção inválida, tente novamente.")

```

```
if __name__ == "__main__":  
    main()
```